

LLM Inference Logging System

Free-Tier Edition — Groq + Gemini + Vercel + Railway + Neon

Free-tier service map

What	Service	Free limit	Sign-up
LLM provider 1	Groq	~14,400 req/day (llama-3.3-70b)	console.groq.com
LLM provider 2	Gemini	1,500 req/day (gemini-2.0-flash)	aistudio.google.com
Frontend	Vercel	Unlimited hobby deploys	vercel.com
Backend API	Railway	\$5 free credit (~500 hrs)	railway.app
Database	Neon	0.5 GB, 1 compute unit	neon.tech
Redis / event bus	Upstash Redis	10,000 commands/day	upstash.com

Architecture overview

The system is split across four free-tier services. The React frontend is deployed on Vercel and communicates with an Express backend on Railway. The backend wraps Groq and Gemini APIs behind a unified SDK, captures inference metadata, stores it via Upstash Redis (event queue), and persists everything to Neon Postgres. A lightweight cron worker inside the Railway app drains the queue every 60 seconds.



Feature implementation prompts

Each feature below is a self-contained prompt you can hand directly to a developer or code generator.

Feature 1**Chatbot UI on Vercel**

Build a React chatbot app deployed to Vercel with:

- Vite + React, no paid dependencies
- Provider selector dropdown: groq and gemini, stored in localStorage
- Multi-turn: maintain messages[] in state, send last 10 turns with each request
- Streaming UI: fetch /api/chat/stream on Railway with ReadableStream, render tokens live
- Conversation sidebar: list sessions, resume and cancel buttons
- Env var: VITE_API_URL=https://your-app.railway.app
- Pass Authorization: Bearer header on every backend call

Feature 2**Multi-provider SDK (Groq + Gemini)**

Build src/lib/LLMClient.ts with a unified interface and two adapters:

- Groq: base URL https://api.groq.com/openai/v1 (OpenAI-compatible), model llama-3.3-70b-versatile, streaming via SSE
- Gemini: use @google/generative-ai, model gemini-2.0-flash, streaming via generateContentStream()
- Both return: { content, provider, model, usage: { promptTokens, completionTokens } }
- Auto retry on 429/503 with exponential backoff, max 3 retries
- Factory: createLLMClient(provider: 'groq' | 'gemini')

Feature 3**Metadata capture + PII redaction**

Build src/lib/withLogging.ts wrapping any LLMClient:

- Record latencyMs, ttftMs (time to first token) using performance.now()
- Assemble InferenceLog: requestId, sessionId, provider, model, timestamp, latencyMs, ttftMs, tokens, previews, status
- PII redact input/output previews: replace emails, phones, card numbers, SSNs with [EMAIL], [PHONE], [CARD], [SSN]
- Set piiDetected: true if any pattern fired
- Fire-and-forget POST to /ingest/log — never await, never delay user response

Feature 4**Streaming response endpoint (Railway)**

Build POST /api/chat/stream in Express:

- Headers: Content-Type: text/event-stream, Cache-Control: no-cache, Connection: keep-alive
- For each token chunk: res.write('data: ' + JSON.stringify({ delta }) + '\n\n')
- On stream end: emit { done: true, usage } then res.end()
- Handle client disconnect: req.on('close', () => abortController.abort())
- Fire inference log asynchronously after stream completion

Feature 5**Ingestion API (Railway)**

Build POST /ingest/log with zod validation:

- Required fields: requestId (UUID), sessionId (UUID), provider (groq|gemini), model, timestamp, latencyMs, status
- On validation fail: 400 { error: 'validation_failed', details: zodError.errors }
- On success: LPUSH inference_queue to Upstash Redis
- Return 202 Accepted immediately — never block on DB write
- Also expose POST /ingest/message for chat message ingestion

Feature 6**Neon Postgres schema**

Three tables — run as migrations on Railway startup:

- conversations: id, session_id (unique), status, provider, model, created_at, message_count, total_tokens
- messages: id, conversation_id (FK), role (user|assistant), content, created_at
- inference_logs: id, request_id (unique), session_id, provider, model, status, latency_ms, ttft_ms, prompt_tokens, completion_tokens, input_preview, output_preview, pii_detected, created_at
- Indexes on: inference_logs(session_id, created_at DESC) and inference_logs(provider, created_at DESC)
- Use @neondatabase/serverless with the pooled connection string

Feature 7**Upstash Redis worker (event bus)**

Lightweight polling consumer using @upstash/redis (stays within 10k/day free limit):

- Ingestion API: LPUSH inference_queue on each log
- Worker endpoint /worker/drain (triggered by node-cron every 60s inside Railway app)
- Drain: LRange inference_queue 0 49 → batch INSERT to Neon → LTRIM inference_queue 50 -1
- Conversation context cache: SET ctx:{sessionId} EX 7200
- Deduplication: SET dedup:{requestId} 1 NX EX 86400 before insert

Feature 8**Observability dashboard (Vercel)**

Build /dashboard route in React using Recharts (free, MIT):

- Fetch GET /api/metrics/summary from Railway — computed via single Neon query with GROUP BY provider
- BarChart: requests by provider; LineChart: latency over time (bucketed by hour); PieChart: error vs success
- Stat cards: total requests, avg latency, error %, total tokens
- Auto-refresh every 30s with useEffect + setInterval
- Neon query: SELECT provider, count(*), avg(latency_ms), sum(total_tokens) FROM inference_logs WHERE created_at > now() - interval '24 hours' GROUP BY provider

Feature 9**Conversation manager (list / resume / cancel)**

Three Railway API routes + Vercel sidebar:

- GET /api/conversations — query conversations ORDER BY last_message_at DESC LIMIT 50
- GET /api/conversations/:sessionId/messages — return all messages for session, ordered by created_at
- DELETE /api/conversations/:sessionId — UPDATE status='cancelled', return 204
- Frontend: render each row with preview, timestamp (date-fns formatDistanceToNow), provider badge, status pill
- Resume: load messages into state, restore full chat thread
- Cancel: window.confirm dialog, optimistic UI update, revert on error

Feature 10**Railway deployment setup**

Configure Express backend for Railway:

- package.json: start: 'node dist/index.js', build: 'tsc'
- railway.toml: builder=nixpacks, healthcheckPath=/health, restartPolicyType=on_failure
- CORS: allow https://your-app.vercel.app origin only in production
- GET /health returns { status: 'ok', providers: ['groq', 'gemini'] }

Required Railway env vars:

- GROQ_API_KEY, GEMINI_API_KEY, DATABASE_URL (Neon pooled), UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN, INGEST_API_KEY, NODE_ENV=production

Feature 11**Vercel deployment setup**

Configure React frontend for Vercel:

- vercel.json: { "rewrites": [{ "source": "/(.*)", "destination": "/" }] }
- Env vars in Vercel dashboard: VITE_API_URL and VITE_INGEST_API_KEY
- All API calls use import.meta.env.VITE_API_URL — never hardcode Railway URL
- Deploy: vercel --prod or connect GitHub repo for auto-deploy on push to main
- Hobby plan: unlimited deployments, custom domain, automatic HTTPS — free forever

.env.example — local development

```
# LLM providers (free)
```

```
GROQ_API_KEY=gsk_your_key_here
```

```
GEMINI_API_KEY=AIzaSy_your_key_here
```

```
# Database – Neon free tier
```

```
DATABASE_URL=postgresql://user:pass@ep-xyz.neon.tech/neondb?sslmode=require
```

```
# Redis – Upstash free tier

UPSTASH_REDIS_REST_URL=https://your-db.upstash.io

UPSTASH_REDIS_REST_TOKEN=your_token_here


# Auth

INGEST_API_KEY=generate-a-random-32-char-string


# Frontend

VITE_API_URL=http://localhost:4000

VITE_INGEST_API_KEY=same-as-above
```

Free API key sign-up links

Service	URL	Notes
Groq	console.groq.com/keys	free, no credit card
Gemini	aistudio.google.com/apikey	free, 1,500 req/day
Neon	neon.tech	free Postgres, instant setup
Upstash	console.upstash.com	free Redis, 10k cmd/day
Railway	railway.app	\$5 free credit, no expiry
Vercel	vercel.com	free forever for hobby

Schema design decisions

Single Postgres instance (Neon) — Keeps the free tier simple. TimescaleDB is skipped since Neon's free tier is plain Postgres — time-bucket queries work with `DATE_TRUNC` instead.

Upstash Redis as lightweight queue — 10k commands/day is plenty for dev traffic. A `LPUSH/LRANGE/LTRIM` pattern replaces a full Kafka/Redis Streams setup while staying free.

`inference_logs.request_id` UNIQUE — Guarantees idempotent inserts — if the worker retries a batch, duplicate requestIds are silently ignored via `ON CONFLICT DO NOTHING`.

PII redaction before storage — `input_preview` and `output_preview` are redacted at the SDK layer before ever hitting the network. Raw content is never stored.

No TimescaleDB on free tier — Replaced with standard Postgres indexes on (`provider`, `created_at` DESC). Aggregation queries use `DATE_TRUNC('hour', created_at)` which is fast enough for small volumes.

Tradeoffs on the free tier

- Railway \$5 credit runs out after ~500 compute hours — fine for a demo, needs a paid plan for persistent production use.
- Upstash 10k commands/day limits throughput. Under heavy load, logs are dropped rather than queued — acceptable for a prototype.
- Neon 0.5 GB fills quickly if inference logs are verbose. The 200-char preview cap on input/output keeps row sizes small.
- No persistent Redis Streams consumer groups — the cron-drain pattern is simpler but has a small race window if Railway restarts mid-drain.
- Groq rate limits (14,400 req/day) reset at midnight UTC. Build in a fallback to Gemini when Groq returns 429.

What to improve with more time

- Add TimescaleDB (use Timescale Cloud free tier) for proper time-series hypertables and percentile queries.
- Replace the cron-drain pattern with Redis Streams consumer groups for true at-least-once delivery.
- Add Docker Compose for one-command local dev with Postgres + Redis containers.
- Kubernetes manifests (k8s/) with HorizontalPodAutoscaler on the ingestion service.
- DeepSeek and Grok adapters in the SDK for full multi-provider coverage.
- A proper DLQ table in Postgres for failed ingestion events with exponential backoff retry worker.
- End-to-end test suite using Vitest + MSW to mock Groq/Gemini APIs.
- GitHub Actions CI/CD pipeline: test → build → deploy to Vercel and Railway on push to main.